Data Science in the Command Line

Why the Command Line?

The command line matters because much of real data science happens on systems where a GUI is unavailable or simply slower than a shell.

Industry reality

- Production jobs often run on Linux servers with no graphical interface
- Cloud platforms, scheduled jobs, and remote systems are shell-first environments

Advantages : Speed, automation, reproducibility, and remote access

The Unix Philosophy

Unix became influential because it encouraged a simple and durable design mindset that still fits modern data workflows.

Core principles

- Do one thing well
- Make programs work together
- Treat text streams as a common interface

Why this still matters

- Composability is still one of the best ways to build reliable systems

Unix Philosophy and Modern Engineering

The Unix philosophy is not just historical trivia; it explains why command-line ecosystems remain so effective.

Modern parallels

- Microservices and modular software both rely on small focused components
- Data workflows are easier to reason about when each step has a single job

Unix Pipeline Example

A short pipeline illustrates the philosophy better than a long definition ever could.

Example:

```
cat data.csv | grep "2026" | cut -d"," -f2 | sort | uniq -c
```

What each step does

```
Read rows | filter | extract a field | sort values| count unique results
```

Each command. is simple, but the combined workflow does real analysis

The shell: why is it important??

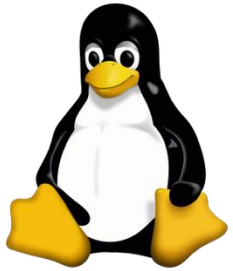
To introduce the shell means to have a discussion about the structure of the computer, operating systems, file systems and history

The shell offers programmatic access to a computer's underlying parts, providing the ability to "do" data analysis on directories, on processes, and on their networks
as there are many flavors of the shell.

It is the first time we come to a decision about choosing 'tools', about evaluating which shell is best for you and the shifting terrain of software development.

Operating Systems

An operating system is a piece of software that organizes and controls hardware and other software, so your computer behaves in a flexible but predictable way. Maybe obviously to us all, Window, MacOS and Linux are those most commonly used for home computers. iOS and Android are most common on mobile devices



A history

In 1964, Bell Labs partnered with MIT and GE to create Multics (Multiplexed Information and Computing Service)

“Such systems must run continuously and reliably 7 days a week, 24 hours a day in a way similar to telephone or power systems, and must be capable of meeting wide service demand from multiple man-machine interaction to the sequential processing of absentee-user jobs, from the use of the system with dedicated languages and subsystems to the programming of the stem itself”

Bell Labs pulled out of the Multics project in 1969 where they eventually started work on a uniplexed information and computing system) because initially it could only support one user; as the system matured it was named Unix,

Dennis Ritchie simply says that Unix is a ‘somewhat treacherous pun on Multics’

The kernel and the shell

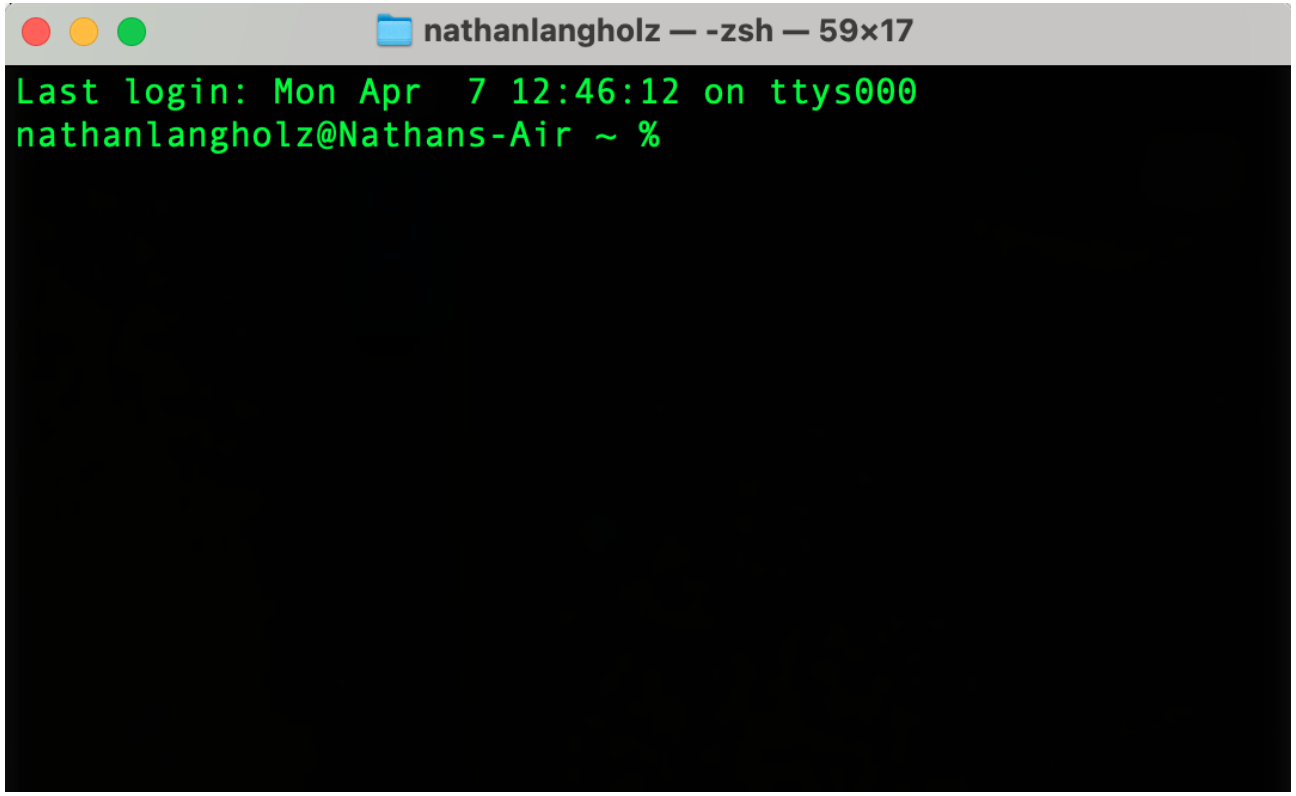
the Unix kernel is the part of the operating system that carries out basic functions like accessing files, handing communication, and others

the Unix shell is a user interface to the kernel

- A shell is a type of program called an interpreter, think of it as a text-based interface to the kernel
- It operates in simple loop: it accepts a command, interprets it, executes the command and waits for another
- The shell displays a prompt to tell you that it is ready to accept a command

Not much going on but lots of potential!

The % is the prompt



Unix Shell(s)

There are, in fact, many different kinds of Unix shells

The table on the right lists a few of the most popular

Bourne shell `/bin/sh`
The oldest and most standardized shell. Widely used for system startup files (scripts run during system startup). Installed in Mac OS X.

Bash (Bourne Again SHell) `/bin/bash`
Bash is an improved version of sh. Combines features from csh, sh, and ksh. Very widely used, especially on Linux systems. Installed in Mac OS X.
http://www.gnu.org/manual/bash/

C shell `/bin/csh`
Provides scripting features that have a syntax similar to that of the C programming language (originally written by Bill Joy). Installed in Mac OS X.

Korn shell `/bin/ksh`
Developed at AT&T by David Korn in the early 1980s. Ksh is widely used for programming. It is now open-source software, although you must agree to AT&T's license to install it. http://www.kornshell.com

TC Shell `/bin/tcsh`
An improved version of csh. The *t* in tcsh comes from the TENEX and TOPS-20 operating systems, which provided a command-completion feature that the creator (Ken Greer) of tcsh included in his new shell. Wilfredo Sanchez, formerly lead engineer on Mac OS X for Apple, worked on tcsh in the early 1990s at the Massachusetts Institute of Technology.

Z shell `/bin/zsh`
Created in 1990, zsh combines features from tcsh, bash, and ksh, and adds many of its own. Installed in Mac OS X.
http://zsh.sourceforge.net

Why the choices?

A shell program was originally meant to take commands, interpret them and then execute some operation

Inevitably, one wants to collect a number of these operations into programs that execute compound tasks at the the same time you want to make interaction on the command line as easy as possible (a history mechanism, editing capabilities and so on)

The original Bourne shell is ideal for programming; the C-shell and its variants are good for interactive use; the Korn shell is a combination of both.

Steve Bourne, creator of sh

File System Structure

Unix uses a single directory tree that starts at one root instead of multiple drive letters.

- Key directories
  - / is root; home stores user folders; usr stores programs; var stores changing data
  - etc is for configuration and tmp is for temporary files
- Data science relevance
  - Clear path discipline makes scripts portable, predictable, and less fragile

Absolute vs Relative Paths

Understanding path types is one of the easiest ways to prevent shell mistakes.

- Absolute paths
  - Start from root, like /home/user/project/data.csv
- Relative paths
  - Start from where you are now, like ../data/raw.csv
- Why this matters
  - Scripts often fail because they assume the wrong working directory

File System Shortcuts

A few path shortcuts make shell navigation much faster once they become second nature.

- Shorthand
  - ~ means home, . means current directory, .. means parent directory
- Why they matter
  - Most broken shell commands are path mistakes more than syntax mistakes
- Practical habit
  - Learn these well before writing larger scripts

Essential Navigation Commands

These are the commands you will use repeatedly until they become muscle memory.

Everyday commands
- pwd, cd, ls, ls -la, and ls -lh

Helpful moves
- cd ~ goes home, cd .. goes up, and cd - jumps back to the last location

Best habit: Verify where you are before running anything destructive

Finding Files with find

Once projects get large, find becomes more useful than clicking through folders manually.

- Common patterns
  - find . -name "*.csv"
  - find . -type f -mtime -7
  - find . -size +100M
- Why it matters
  - Search by name, time, or size without leaving the terminal

Navigation Example

A short interactive session is enough to establish good terminal habits.

- Example session
  - pwd
  - cd week-2/examples
  - ls -la
- Interpretation
  - Confirm location, move intentionally, then inspect before acting

File Operations

Once you can move around, the next step is creating, copying, renaming, and deleting safely.

- Create and organize
  - touch file.txt, mkdir data, mkdir -p data/raw/2026
- Copy, move, remove
  - cp copies, mv moves or renames, rm deletes, rm -r removes folders recursively
- Important caution
  - rm has no undo, so caution beats speed

Name of command

How hard the computer is thinking about it
Process ID
Result of the command top; this is a printout of all the processes running on your computer

nathanlangholz — top — 94×41

```
Processes: 673 total, 3 running, 670 sleeping, 3054 threads                    14:37:08
Load Avg: 3.58, 3.27, 3.37  CPU usage: 14.31% user, 7.4% sys, 78.63% idle
SharedLibs: 423M resident, 70M data, 16M linkedit.
MemRegions: 0 total, 0B resident, 1360K private, 747M shared.
PhysMem: 8079M used (2828M wired, 1238M compressor), 240M unused.
VM: 39T vsize, 5047M framework vsize, 168007066(886) swapins, 179820842(0) swapouts.
Networks: packets: 33358385/38G in, 13958509/13G out.
Disks: 35008277/1131G read, 14719577/855G written.

PID    COMMAND      %CPU  TIME     #TH   #WQ  #PORT MEM    PURG   CMPRS  PGRP  PPID  STATE
155    WindowServer 23.2  13:57:40 14    6    4853  898M-  11M    219M-  155   1     sleeping
194    coreaudiod   10.2  51:11.67 12    4    2197  21M-   0B     11M-   194   1     sleeping
41829  top          9.2   00:09.40 1/1   0    31    9032K  0B     1036K  41829 41793 running
41677  com.apple.We 7.8   05:49.60 18/1  8    124   1487M+ 9676K  399M-  41677 1     running
0      kernel_task  4.8   10:06:26 261/4 0    0     588M-  0B     0B     0     0     running
11099  Google Drive 4.6   15:54:57 9     1    140   37M    0B     19M    1552  11095 sleeping
22553  com.apple.We 4.3   19:37.14 38    6    450-  178M-  6784K  149M   22553 1     sleeping
11103  Google Drive 3.0   09:13:04 12    1    332   27M    0B     24M    1552  11095 sleeping
22429  com.apple.Ap 2.2   17:18.66 3     2    300   776K   0B     272K   22429 1     sleeping
29078  com.apple.We 1.4   03:05.32 9     1    118-  253M-  0B     188M-  29078 1     sleeping
1474   Spotlight    1.4   03:02.20 5     2    503+  88M    0B     62M-   1474  1     sleeping
22493  Safari       1.1   38:14.86 17    9    3001- 357M-  28K    299M-  22493 1     sleeping
40548  com.apple.We 1.0   03:46.15 11    4    108   503M   0B     435M-  40548 1     sleeping
1468   Terminal     1.0   01:02.29 11    4    540   68M+   1792K  23M-   1468  1     sleeping
1558   Google Chrom 0.8   99:45.84 12    1    139+  64M+   0B     40M-   1464  1464  sleeping
113    mds          0.8   83:50.47 8     5    611   39M    0B     31M-   113   1     sleeping
22425  bluetoothd   0.8   32:33.17 11    5    617+  9412K+ 20K    4740K- 22425 1     sleeping
22498  com.apple.We 0.7   48:21.17 10    5    191-  98M-   264K   38M-   22498 1     sleeping
5768   Google Chrom 0.6   47:45.51 14    1    401   379M+  0B     310M-  1464  1464  sleeping
1464   Google Chrom 0.5   02:08:20 46    3    1746+ 231M   0B     166M-  1464  1     sleeping
41832  screencaptur 0.5   00:00.30 2     1    66    4160K+ 620K   0B     1470  1470  sleeping
566    sharingd     0.2   24:02.25 5     2    495+  22M+   0B     17M-   566   1     sleeping
86     logd         0.2   32:51.95 4     3    2363  22M    0B     27M-   86    1     sleeping
586    useractivity 0.2   01:00.43 4     3    114+  2720K+ 0B     1376K- 586   1     sleeping
483    identityserv 0.2   11:55.14 6     2    826+  21M+   128K   18M-   483   1     sleeping
32877  audioanalyti 0.1   01:05.47 4     3    56    1724K+ 0B     448K-  32877 1     sleeping
166    lsd          0.1   04:20.25 2     1    140+  6136K+ 0B     5304K- 166   1     sleeping
91     fseventsd    0.1   32:59.10 14    1    199   3664K- 0B     1404K  91    1     sleeping
160    runningboard 0.1   36:11.74 7     6    944   7636K  0B     1332K  160   1     sleeping
99     powerd       0.1   12:25.51 4     3    155+  3784K+ 0B     1872K- 99    1     sleeping
29451  com.docker.b 0.1   40:03.69 120   1    437   90M    0B     47M    29430 29430 sleeping
```

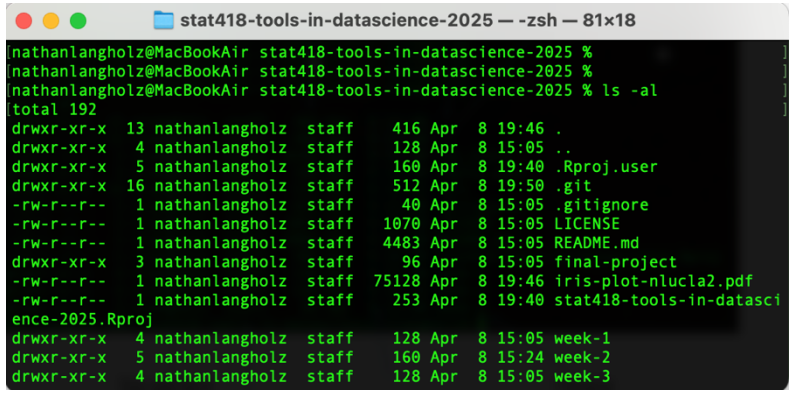
Viewing File Contents

You often do not want to open a file in an editor just to inspect what is inside it.

- Quick inspection tools
  - cat for small files, less for larger files, head and tail for sampling
- Monitoring
  - tail -f follows a file as new lines are appended, which is ideal for logs
- Good workflow
  - Inspect first, then transform

Shorthand for your present working directory

Shorthand for the directory one level above
The file's size in bytes
Read, write, execute permissions
Your username
Filename
The group you belong to that owns the file
The file's creation date

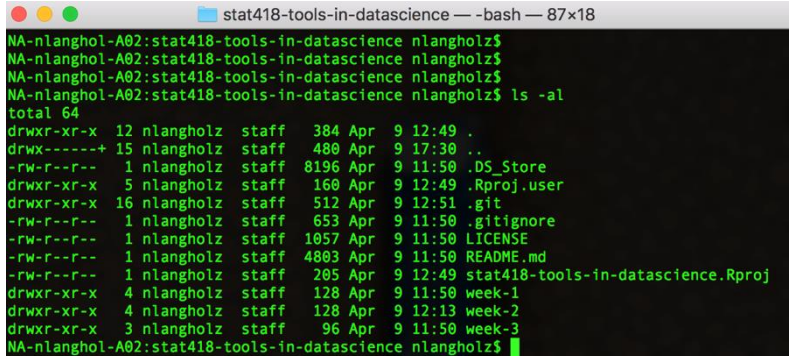


Permission bits

Unix can support many users on a single system and each user can belong to one or more groups
every file in a Unix filesystem is owned by some user and one of that user's groups; each file also has a set of permissions specifying which users can
r: read or w: write (modify) or x: execute
the file; these are specified with three 'bits' and we need three sets of bits to define what the user can do, what their group (that owns the file) can do and what others can do
the command chmod changes the permissions on a file but we will leave that for you to discover on your own

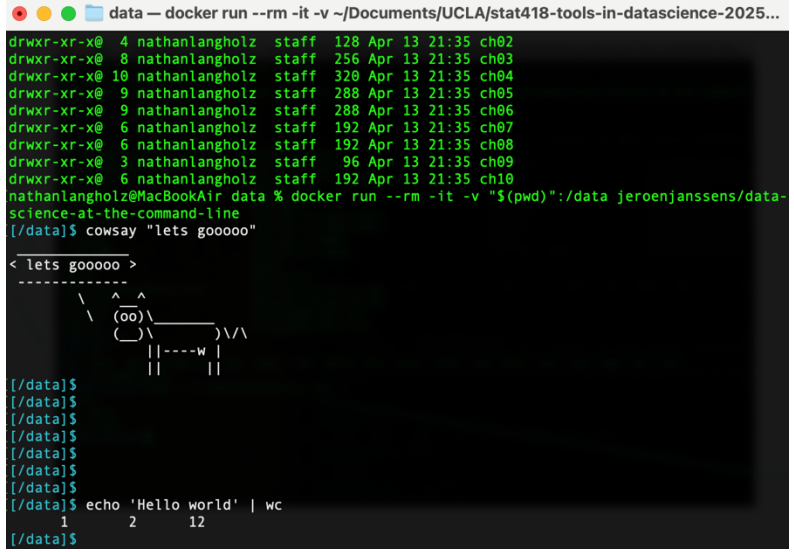
Permission bits

The type of file
What you can do to the file
What the owning group can do
What others can do



Redirecting output with “|”

takes output from one command and submits
it as input to the next command



You can connect pipes and have data stream from process to p

data — docker run --rm -it -v ~/Documents/UCLA/stat418-tools-in-datascience-2025...

```
[/data]$ seq 5
1
2
3
4
5
[/data]$ seq 30 | grep 3
3
13
23
30
[/data]$ seq 200 | grep 3 | wc -l
38
[/data]$
[/data]$
[/data]$
[/data]$
[/data]$
[/data]$
[/data]$
[/data]$
[/data]$
```

Pipes and Redirection

Pipes and redirection are what make the shell feel like a programmable data system.

- Redirection
  - > writes output, >> appends, < reads input, 2> captures errors, &> captures both
- Pipes
  - | sends one command's output directly into the next command
- Practical effect
  - You can build surprisingly complex workflows without writing much code

Redirection Examples

Redirection is how you turn commands into reusable workflows that leave behind useful artifacts.

- Examples
  - python script.py > results.txt
  - grep "ERROR" app.log 2> grep_errors.txt
  - sort input.txt > sorted.txt
- Practical value
  - Capturing output and errors separately makes debugging much easier

Pipe Example: CSV Analysis

A pipeline can answer a concrete data question in seconds with no spreadsheet needed.

- Example
  - grep "2026" sales.csv | cut -d"," -f3 | sort -n | tail -10
- What happens
  - Filter rows, isolate a field, sort the values, and keep the largest results
- Why this is worth learning
  - Fast answers matter when you are iterating on real work

Text Processing: grep

grep is one of the most useful command-line tools because searching text is a universal task.

- What it does
  - Finds lines that match a pattern
- Common flags
  - -i ignores case, -v inverts matches, -c counts, -n shows line numbers, -r searches recursively
- Common uses
  - Logs, source code, CSV files, notebooks, and plain-text corpora

grep Coding Examples

A few grep patterns cover a surprising amount of real work.

- Basic examples
  - grep "pattern" file.txt
  - grep -i "pattern" file.txt
  - grep -r "TODO" .
- Regex examples
  - grep "^Error" logfile.txt
  - grep "Error$" logfile.txt
  - grep "[0-9]\{3\}-[0-9]\{4\}" contacts.txt

Read the man and help pages!

If the command uniq is unfamiliar you can look up its usage
or

```
man uniq
uniq --help
```

Real Example: Huckleberry Finn

Project Gutenberg is a nice reminder that plain text can be real data, not just toy examples.

- Download text
  - curl -o huck_finn.txt https://www.gutenberg.org/files/76/76-0.txt
- Analyze quickly
  - wc -l huck_finn.txt
  - grep -i "river" huck_finn.txt | wc -l
  - grep -in "river" huck_finn.txt | head -5

Text Processing: sed

sed is a stream editor, which means it transforms text as it flows through a command.

- Best-known use
  - Search and replace with s/old/new/g
- Other uses
  - Delete matching lines, print selected ranges, and clean repetitive noise
- Why it is handy
  - Great for quick transformations when opening an editor is overkill

sed Coding Examples

The syntax looks strange at first, but a few patterns cover most everyday needs.

- Replace text
  - sed "s/old/new/" file.txt
  - sed "s/old/new/g" file.txt
  - sed "s/old/new/gi" file.txt
- Delete or extract
  - sed "/pattern/d" file.txt
  - sed -n "5,10p" file.txt

Text Processing: awk

awk is ideal when your text already has structure such as columns or delimiters.

- Strengths
  - Select fields, filter rows, and compute numeric summaries
- CSV-friendly
  - Use -F "," to work with comma-separated files
- Why data scientists care
  - It gives quick answers before you ever reach for pandas

awk Coding Examples

A few awk one-liners go a long way when the data is already tabular.

- Column selection
  - awk -F"," "{print $2}" data.csv
  - awk "{print $1, $3}" file.txt
- Filtering and aggregation
  - awk -F"," "$2 > 25 {print $1, $3}" people.csv
  - awk "{sum += $1} END {print sum}" numbers.txt
  - awk "{sum += $1; count++} END {print sum/count}" numbers.txt

grep vs sed vs awk

These tools are easiest to remember when you think of them as different types of text work.

- grep
  - Find matching lines
- sed
  - Transform or delete text in a stream
- awk
  - Work with fields, filters, and calculations

Building Data Pipelines

Most command-line analysis can be understood as a sequence of small operations chained together.

- Basic pattern
  - Collect → filter → transform → summarize
- Common tools
  - grep, cut, sort, uniq, tr, wc, head, and tail
- Why this scales
  - Small pieces are easier to test, debug, and reuse than monolithic scripts

Pipeline Example: Word Frequency

Word frequency is a great example because it looks complicated but is built from simple steps.

- Workflow
  - cat huck_finn.txt | tr "[:upper:]" "[:lower:]"
  - tr -s "[:space:]" "\n" | grep -v "^$" | sort | uniq -c
  - sort -rn | head -20
- Outcome
  - A complete text-analysis pipeline in one command sequence

Bash Scripting Basics

At some point, typing commands interactively becomes slower than saving them into a script.

- Core idea
  - A bash script is a file containing commands you want to run together
- Standard patterns
  - Start with #!/bin/bash, accept arguments, validate inputs, quote variables
- Payoff
  - Repeated work becomes a reusable tool and a record of your workflow

Running Scripts and Permissions

Shell scripts are files, so the system needs both an interpreter and permission to execute them.

- Common pattern
  - chmod +x analyze_text.sh
  - ./analyze_text.sh huck_finn.txt
- Why it matters
  - Many “it does not run” problems are really permission or path issues

Script Example: Text Analyzer

A small script can already demonstrate the full value of automation.

- What it checks
  - Confirms a filename argument is present and the file exists
- What it computes
  - Counts lines, words, characters, and prints common words
- Why it is useful
  - A repeatable analysis beats manually retyping a pipeline

Variables and Command Substitution

Variables make scripts easier to read and much easier to change later.

- Variables
  - NAME="Alice" and OUTPUT_DIR="results"
- Reading values
  - echo $NAME
- Command substitution
  - CURRENT_DATE=$(date +%Y-%m-%d)
  - FILE_COUNT=$(ls *.txt | wc -l)

Loops in Bash

Loops are what let a shell script process many files without repeated copy-paste.

- for loops
  - for file in *.csv; do ... done
- while loops
  - while [$count -lt 5]; do ... done
- Why they matter
  - Batch processing is one of the shell's biggest strengths

Functions in Bash

Functions keep larger scripts from collapsing into one long unreadable block of commands.

- Benefits
  - Organize code, avoid repetition, and make intent clearer
- Pattern
  - analyze_file() { local file=$1; ... }
  - process_csv() { local input=$1; local output=$2; ... }
- Deeper point
  - Small focused functions bring the Unix philosophy into scripting

Error Handling and Debugging

Reliable scripts are usually simple scripts plus a few excellent defensive habits.

- Best-practice default
  - set -euo pipefail
- Debugging help
  - bash -x script.sh prints commands as they execute
- Testing mindset
  - Try bad inputs on purpose before trusting a workflow

Robust Script Example

Real scripts need to fail clearly when something important goes wrong.

- Better pattern
  - Check downloads, trap failures, and write errors to stderr
- Example
  - if ! curl -f -o "$output" "$url"; then echo "download failed" >&2; fi
- Goal
  - Fail loudly instead of silently producing bad downstream results

Real-World Pipeline Script

A realistic script usually follows the same four-stage structure again and again.

- Typical stages
  - Configure, collect, process, and report
- Example behavior
  - Download files, loop through them, count matches, and write a summary report
- Why it matters
  - This is exactly the pattern you can reuse for Assignment 1

Version Control

Shell (bash) scripts are often the first step toward reproducible computational research. By capturing data processing and analysis steps in executable scripts, we reduce reliance on memory and manual intervention, making it possible to rerun the same workflow on the same machine and obtain consistent results.

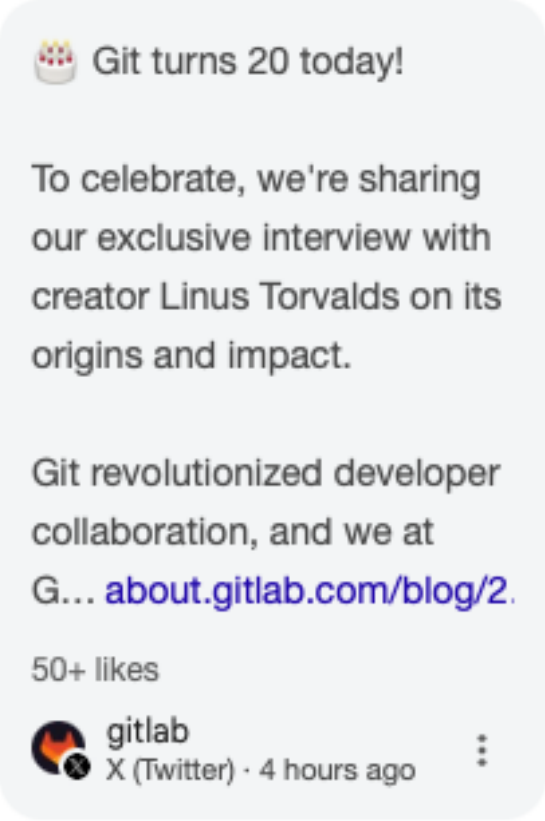
However, scripts alone are not sufficient over time. As analyses evolves, parameters change, bugs are fixed, or methods improve we also need a systematic way to track how those scripts changed and which version produced a given result.

Version control complements scripting by recording the history of scripts, configuration files, and even documentation, making it possible to recover the exact state of an analysis at a specific point in time.

Git: what is it?

A version control system to track the history of changes as people and teams collaborate on code projects together

Git manages the evolution of a set of files (or in our case a data analysis or model development) in a repository - this provides a transparent history of changes, who made them and how each contribute all while reducing redundancy and unnecessary work



04/07/2025

A history

In 2005, Linus Torvalds was developing Linux when BitKeeper (proprietary source-control management) revoked their free license. Torvalds set 4 goals for his development needs for a SCM and when he couldn’t find anything sufficient he decided to create his own. Development began April 3 and was released April 7.

"I'm an egotistical bastard, and I name all my projects after myself. First 'Linux', now ‘Git'".”

('git' is British slang for "pig headed, think they are always correct, argumentative").

git / git

Type / to search

Code | Pull requests 225 | Actions | Security 29 | Insights

git Public

Watch 2417 | Fork 26.1k | Star 54.2k

e83c516 | 6 Branches | 974 Tags | Go to file | Code

Linus Torvalds Initial revision of "git", the information manager from hell — e83c516 · 21 years ago — 1 Commit

Makefile	Initial revision of "git", the information manager from hell	21 years ago
README	Initial revision of "git", the information manager from hell	21 years ago
cache.h	Initial revision of "git", the information manager from hell	21 years ago
cat-file.c	Initial revision of "git", the information manager from hell	21 years ago
commit-tree.c	Initial revision of "git", the information manager from hell	21 years ago
init-db.c	Initial revision of "git", the information manager from hell	21 years ago
read-cache.c	Initial revision of "git", the information manager from hell	21 years ago
read-tree.c	Initial revision of "git", the information manager from hell	21 years ago
show-diff.c	Initial revision of "git", the information manager from hell	21 years ago
update-cache.c	Initial revision of "git", the information manager from hell	21 years ago
write-tree.c	Initial revision of "git", the information manager from hell	21 years ago

README

```
        GIT - the stupid content tracker

"git" can mean anything, depending on your mood.

 - random three-letter combination that is pronounceable, and not
   actually used by any common UNIX command.  The fact that it is a
   mispronounciation of "get" may or may not be relevant.
 - stupid. contemptible and despicable. simple. Take your pick from the
   dictionary of slang.
 - "global information tracker": you're in a good mood, and it actually
   works for you. Angels sing, and a light suddenly fills the room.
 - "goddamn idiotic truckload of sh*t": when it breaks

This is a stupid (but extremely fast) directory content manager.  It
doesn't do a whole lot, but what it _does_ do is track directory
contents efficiently.
```

About

Git Source Code Mirror - This is a publish-only repository but pull requests can be turned into patches to the mailing list via GitGitGadget (https://gitgitgadget.github.io/). Please follow Documentation/SubmittingPatches procedure for any of your improvements.

c | shell | hacktoberfest

Readme
View license
Code of conduct
Security policy
Activity
Custom properties
54.2k stars
2.4k watching
26.1k forks
Report repository

Releases

974 tags

Packages

No packages published

Contributors 1,746

+ 1,732 contributors

https://github.com/git/git/tree/e83c5163316f89bfbde7d9ab23ca2e25604af290

Git

I< < Prev Random Next > >I

I< < Prev Random Next > >I

Permanent link to this comic: https://xkcd.com/1597/
Image URL (for hotlinking/embedding): https://imgs.xkcd.com/comics/git.png

Repository management services

How Git Works

Git feels much simpler once you think of it as a workflow between three states.

Three states
- Working directory
- Staging area
- Repository history

Standard flow
- edit → add → commit → push

Git Best Practices

Good commit messages are crucial for understanding project history. Future you and your collaborators will thank you when they need to understand what changed and why.

Commit messages matter
Good: "Add user authentication feature"
Good: "Fix bug in data processing pipeline"
Bad: "Update files" and "Change line 47"

Commit logical units, commit often, avoid huge mixed changes, and use .gitignore for generated files, environments, and secrets

Git Workflow Commands

Most collaborative Git work follows a predictable feature-branch workflow.

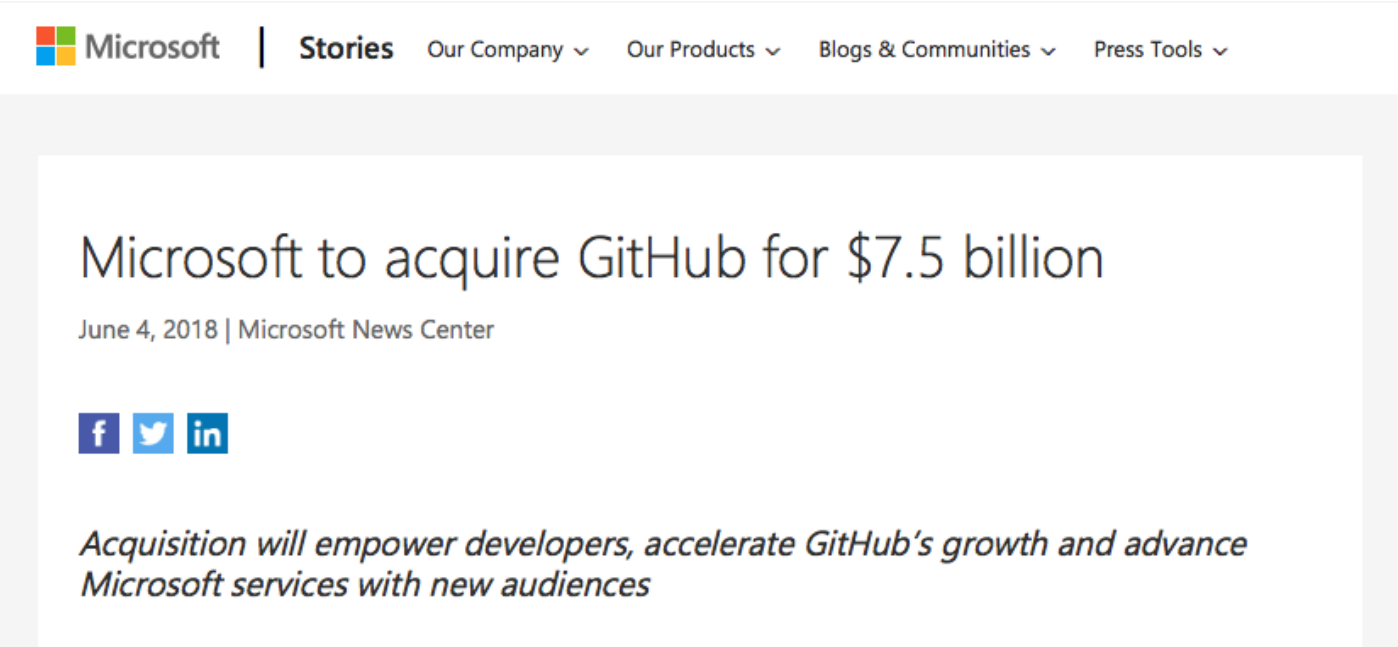
Typical sequence

- git clone ...
- git checkout -b feature/my-analysis
- git add . && git commit -m "Add analysis script"
- git push origin feature/my-analysis

It's not all fun and games. Actually, it's quite hard. Initially just working on solo repositories becomes fine but once collaborating you'll start to run into merge conflicts

Keep going!!

Let it be known…



https://www.theverge.com/2018/6/18/17474284/microsoft-github-acquisition-developer-reaction

GitHub and Pull Requests

Git handles version control locally, but GitHub made collaborative workflows much more approachable.

- What GitHub added
  - Web interfaces, pull requests, issue tracking, and visible project history
- Why pull requests matter
  - They create a reviewable place to discuss changes before merging
- Professional habit
  - Treat pull requests as communication tools, not just merge buttons

A bit more on reproducibility…

There has been a bit of crisis taking place in regards to reproducibility as a whole in science fields. and this true of the data science field as well.

We've seen some of the tools to enable us to do this reproducible work, but what concepts should we be thinking of as we use them?

Repeatability

or test–retest reliability is the variation in measurements taken by a single person or instrument on the same item, under the same conditions, and in a short period of time. A less-than- perfect test–retest reliability causes test–retest variability.

Your systems must be repeatable and reliable, the same query on the same data should return the same results. Otherwise, none of this applies.

Reproducibility

an analysis is **reproducible** if there is a specific set of computational functions/ analyses (usually specified in terms of code) that exactly reproduce all of the numbers in a published paper from raw data. It is now recognized that a critical component of the scientific process is that data analyses can be reproduced.

same query on the same data should return the same results. Otherwise, none of this applies

Replicability

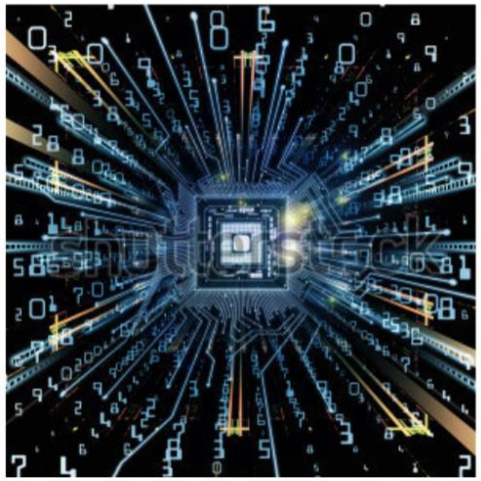
but just because a study is reproducible does not mean that it is replicable. **Replicability** is stronger than **reproducibility**. A study is only **replicable** if you perform the exact same experiment (at least) twice, collect data in the same way both times, perform the same data analysis, and arrive at the same conclusions.

same query on the same data should return the same results. Otherwise, none of this applies

Stodden's Taxonomy of Reproducibility

Computational

when detailed information is provided about code, software, hardware and implementation details.

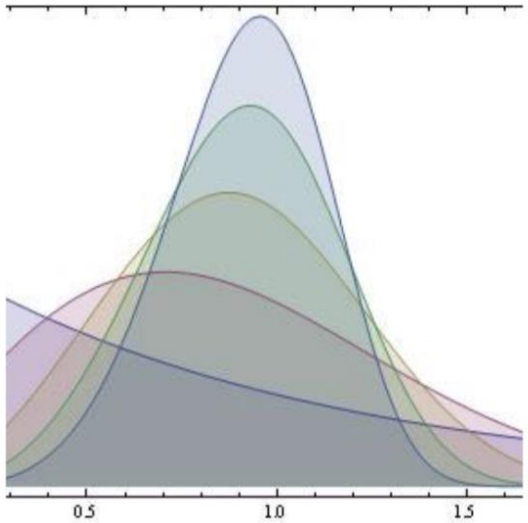


Empirical

when detailed information is provided about code, software, hardware and implementation details.

Statistical

when detailed information is provided about code, software, hardware and implementation details.



Computational reproducibility should be…

Reviewable

The descriptions of the research methods can be independently assessed and the results judged credible

Replicable

tools are made available that would allow one to duplicate the results of the research, for example by running the authors' code to produce the plots shown in the publication.

Auditable

sufficient records (including data and software) have been archived so that the research can be defended later if necessary or differences between independent confirmations resolved. The archive might be private, as with traditional laboratory notebooks.

Confirmable

the main conclusions of the research, or the model in production can be attained independently without the use of software provided by the author.

Simple rules for reproducible computational research

track results - whenever a result may be of potential interest, keep track of how it was produced. as a minimum, you should at least record sufficient details on programs, parameters, and manual procedures to allow yourself, in a year or so, to approximately reproduce the results.

script everything - whenever possible, rely on the execution of programs instead of manual procedures to modify data. If manual operations cannot be avoided, you should as a minimum note down which data files were modified or moved, and for what purpose.

store program versions - In order to exactly reproduce a given result, it may be necessary to use programs in the exact versions used originally. As a minimum, you should note the exact names and versions of the main programs you use.

Simple rules for reproducible computational research

use version control - even the slightest change to a computer program can have large intended or unintended consequences. As a minimum, you should archive copies of your scripts from time to time, so that you keep a rough record of the various states the code has taken during development

store data & intermediate results - in principle, as long as the full process used to produce a given result is tracked, all intermediate data can also be regenerated. In practice, having easily accessible intermediate results may be of great value. As a minimum, archive any intermediate result files that are produced when running an analysis

set a random number seed- many analyses and predictions include some element of randomness, meaning the same program will typically give slightly different results every time it is executed. As a minimum, note which analysis steps involve randomness, so that a level of discrepancy can be anticipated when reproducing the results.

Simple rules for reproducible computational research

store data viz inputs - from the time a figure is first generated to it being part of an analysis. It is critical to store the data and process that generated it. As a minimum, one should note which data formed the basis of a given plot and how this data could be reconstructed.

allow levels of analysis - in order to validate and fully understand the main result, it is oLen useful to inspect the detailed values underlying the summaries. Make those fluid and explorable, as a minimum at least once generate, inspect, and validate the detailed values underlying the summaries.

generate text programmatically- there is nothing quite as embarrassing as having a disagreement between your analytical narrative in a writeup and having the tables and figures disagree. Connect them so that there is no change of disagreement.

.